

FORWARD DEPLOYED AI ENGINEERING

Capstone Project

Setup Guide

Getting your environment working on day one

Work through this on the first day. Every hour you delay it becomes an hour lost later, usually at the least convenient moment.

REFERENCE · WEEK ONE, DAY ONE

What this document contains

- 01 Before you begin** What you need installed already
- 02 The Python environment** Virtual environment and dependencies
- 03 Model access** Getting and storing your API key safely
- 04 The vector store** Chroma and your first embeddings
- 05 The decision log database** SQLite or PostgreSQL
- 06 Monitoring** Prometheus and Grafana
- 07 Continuous integration** GitHub Actions
- 08 Project structure** Where everything should live
- 09 When things go wrong** The errors you are most likely to hit

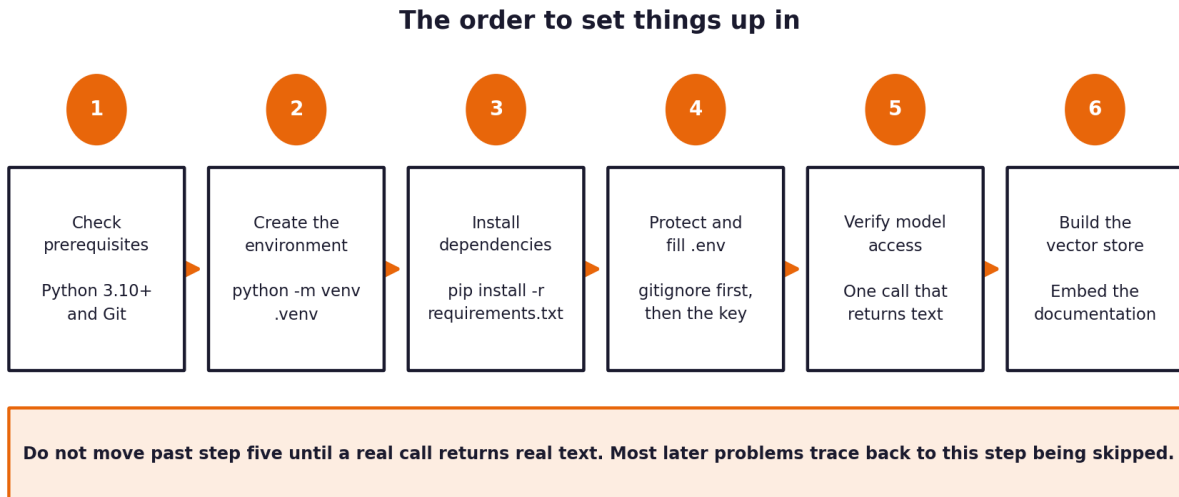


Figure 1. The order to work through this in. Do not skip step five.

01 Before you begin

Check these four things before you start. Sorting them out now takes ten minutes; discovering one of them is missing halfway through the setup takes considerably longer.

What you need	How to check	If it is missing
Python 3.10 or later	<code>python3 --version</code>	Install from python.org or your package manager. Earlier versions will fail on parts of the stack.
Git	<code>git --version</code>	Install from git-scm.com . You need it for version control and for the assessment of your commit history.
A code editor	—	Anything you are comfortable with. VS Code is a reasonable default.
About 4 GB of free disk space	—	The embedding model and the vector store need room.

02 The Python environment

Always work inside a virtual environment. Installing these packages system-wide will eventually break something else on your machine, and it makes your project impossible for anyone else to reproduce.

Creating the environment

```
# Create it once, in your project root
python3 -m venv .venv

# Activate it. On macOS or Linux:
source .venv/bin/activate

# On Windows:
.venv\Scripts\activate

# Confirm you are inside it. The path should point at .venv
which python
```

You will need to activate the environment in every new terminal session. If you find yourself confused about why an import is failing, the first thing to check is whether the environment is active.

Installing the dependencies

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

The package list is in the configuration folder of this pack. Copy it into your project root before running the command above. If a package fails to build, the cause is almost always a Python version below 3.10.

03 Model access

You need access to a language model through an API. OpenRouter and Groq both offer free tiers that are sufficient for this project. Create an account with either, generate a key, and then read the next paragraph carefully before you do anything with it.

Never commit your key

An API key committed to a repository is compromised, even if you delete it in a later commit, because it remains in the history. Your submission is scanned for credentials and any found are treated as a serious finding. Keys live in a .env file, and .env goes in .gitignore before you write anything into it.

Setting it up correctly

```
# First, protect the file before it exists
echo ".env" >> .gitignore

# Then create it from the template
cp .env.example .env

# Then edit .env and fill in your key
```

Your .env file should look like this, with your own value in place of the placeholder:

```
OPENROUTER_API_KEY=your_key_here
MODEL_NAME=meta-llama/llama-3.1-8b-instruct
EMBEDDING_MODEL=all-MiniLM-L6-v2
CHROMA_PATH=./storage/chroma
DATABASE_URL=sqlite:///./storage/decisions.db
LOG_LEVEL=INFO
CONFIDENCE_THRESHOLD=0.80
```

Reading it in code

```
import os
from dotenv import load_dotenv

load_dotenv()

API_KEY = os.environ["OPENROUTER_API_KEY"] # fails loudly if absent, which is what you want
MODEL = os.getenv("MODEL_NAME", "meta-llama/llama-3.1-8b-instruct")
```

Checking it works

Do not move on until this returns a response. A great many later problems turn out to be an environment variable that was never actually loaded.

```
import os, requests
```

```
from dotenv import load_dotenv
load_dotenv()

response = requests.post(
    "https://openrouter.ai/api/v1/chat/completions",
    headers={"Authorization": f"Bearer {os.environ['OPENROUTER_API_KEY']}"},
    json={"model": os.environ["MODEL_NAME"],
        "messages": [{"role": "user", "content": "Reply with the word ready."}]},
    timeout=30,
)
print(response.status_code)
print(response.json())
```

04 The vector store

Chroma runs locally and stores its data on disk, so there is no service to provision. Load the documentation file from the datasets folder, split it into passages, embed them and store them.

```
import json
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter

with open("data/documentation.json") as f:
    documents = json.load(f)

splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=120)

texts, metadatas = [], []
for doc in documents:
    for chunk in splitter.split_text(doc["content"]):
        texts.append(chunk)
        metadatas.append({"doc_id": doc["id"], "title": doc["title"]})

embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
store = Chroma.from_texts(
    texts=texts, metadatas=metadatas, embedding=embeddings,
    persist_directory="./storage/chroma",
)
store.persist()
print(f"stored {len(texts)} passages")
```

Chunk size is a design decision, not a default

The values above are a starting point. Chunks that are too large retrieve irrelevant material alongside the relevant part; chunks that are too small lose the context that made the passage meaningful. Try at least two configurations, measure the difference in retrieval quality, and record which you chose and why. This is exactly the kind of decision the assessment is looking for.

05 The decision log database

SQLite is entirely adequate at this scale and requires no setup beyond a file path. Use PostgreSQL only if you have a specific reason.

```
import sqlite3

connection = sqlite3.connect("./storage/decisions.db")
connection.execute("""
    CREATE TABLE IF NOT EXISTS decisions (
        decision_id    TEXT PRIMARY KEY,
        created_at      TEXT NOT NULL,
        ticket_id       TEXT NOT NULL,
        stage           TEXT NOT NULL,
```

```
        prediction      TEXT,  
        confidence      REAL,  
        threshold       REAL,  
        action_taken    TEXT NOT NULL,  
        reason          TEXT NOT NULL,  
        sources_used    TEXT,  
        guardrails      TEXT,  
        prompt_version  TEXT,  
        requirement_ids TEXT  
    )  
""")  
connection.commit()
```


06 Monitoring

You need to be able to see what your system is doing while it runs. Expose metrics from your application, have Prometheus collect them, and put a Grafana dashboard in front.

Exposing metrics

```
from prometheus_client import Counter, Histogram, start_http_server

TICKETS = Counter("tickets_processed_total", "Tickets processed", ["channel", "outcome"])
LATENCY = Histogram("response_seconds", "End to end response time")
GUARDRAIL = Counter("guardrail_blocks_total", "Responses blocked", ["guardrail"])

start_http_server(8001) # metrics available at localhost:8001/metrics

# then, in your handler
with LATENCY.time():
    result = process(ticket)
TICKETS.labels(channel=ticket.channel, outcome=result.action).inc()
```

Prometheus configuration

```
# prometheus.yml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: "support-system"
    static_configs:
      - targets: ["localhost:8001"]
```

What your dashboard should show

- Tickets processed per hour, split by channel and by outcome.
- The proportion answered automatically against the proportion escalated.
- Response latency at the median and the ninety-fifth percentile.
- Guardrail activations, by guardrail, over time.
- The distribution of confidence scores, which is where drift shows up first.

07 Continuous integration

A pipeline that runs your tests on every push. It is not optional in the assessment, and it takes about fifteen minutes to set up.

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
```

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: python -m pip install --upgrade pip
      - run: python -m pip install -r requirements.txt
      - run: python -m pytest tests/ -v
```

Do not put your key in the workflow file

Tests that need model access should either use a recorded response or read the key from GitHub's encrypted secrets. A key pasted into a workflow file is public the moment you push.

08 Project structure

Use this layout. It matches what the submission guide requires, so adopting it now saves you a reorganisation in week three.

your-project/	
README.md	setup and run instructions
requirements.txt	pinned dependencies
.env.example	every variable, placeholder values only
.gitignore	must include .env and storage/
src/	
ingest.py	normalise tickets from all four channels
classify.py	intent and urgency with confidence
retrieve.py	vector search over the documentation
route.py	the escalation decision and its threshold
generate.py	answer drafting with citations
guardrails.py	the checks that can block a response
logging_store.py	the decision log
api.py	the FastAPI application
prompts/	
build/	prompts that run inside the system
evaluation/	prompts used to judge output
README.md	the register, with versions
tests/	
evaluation/	
harness.py	runs the hidden evaluation set end to end
results/	dated output from each run
docs/	
architecture.md	
data/	small samples only
storage/	generated at runtime, never committed
.github/workflows/ci.yml	

09 When things go wrong

These account for the large majority of setup problems reported by previous cohorts.

What you see	What it usually means	What to do
ModuleNotFoundError on a package you installed	The virtual environment is not active in this terminal.	Activate it and check that `which python` points inside .venv.
KeyError on an environment variable	load_dotenv() was not called, or .env is not in the working directory.	Call load_dotenv() before reading any variable, and check where you launched the process from.
A 401 from the model provider	The key is wrong, expired, or has a stray newline from being pasted.	Print the length of the key. If it is longer than expected, you have whitespace in it.
Chroma returns no results	The store was written to a different directory than the one you are reading.	Use an absolute path from your configuration in both places.
Retrieval returns plausible but wrong passages	Your chunks are too large, so relevant text is diluted by surrounding material.	Reduce chunk size, increase overlap slightly, and measure the difference.
Very slow embedding on first run	The model is being downloaded.	Expected. It is cached afterwards. Do this once at the start, not during a demonstration.
Tests pass locally and fail in CI	Something works because of a file or variable that exists only on your machine.	That is precisely what CI is for. Fix the dependency rather than skipping the test.

The check that saves the most time

At the end of your first day, clone your own repository into a fresh directory and follow your own README from the first line. Whatever is missing will surface immediately, and it will be far less painful now than in week three.